

Object.keys, values, entries – Simple Hinglish Summary

Source: <https://javascript.info/keys-values-entries>

Main baat

Ab individual data structures se hatkar unpar **iteration** ki baat karte hain.

Pichhle chapter me humne `map.keys()`, `map.values()`, `map.entries()` dekhe. Ye methods **generic** hain, aur data structures ke liye inhe use karne ka common agreement hai. Agar hum apna data structure banayein to unhe bhi ye implement karne chahiye.

Ye in ke liye supported hain:

- `Map`
- `Set`
- `Array`

Plain objects ke liye bhi aise hi methods hain, lekin **syntax thoda alag** hai.

1. `Object.keys`, `Object.values`, `Object.entries`

Plain objects ke liye ye methods hain:

- `Object.keys(obj)`: keys ka array return karta hai
- `Object.values(obj)`: values ka array return karta hai
- `Object.entries(obj)`: `[key, value]` pairs ka array return karta hai

`Map` se farak:

	<code>Map</code>	<code>Object</code>
Call syntax	<code>map.keys()</code>	<code>Object.keys(obj)</code> , <code>obj.keys()</code> nahi
Kya return karta hai	iterable	"asli" Array

Pehla farak: `Object.keys(obj)` likhna padta hai, `obj.keys()` nahi. **Kyu?** Flexibility ke liye. Yaad rakho, objects JS me saare complex structures ka base hain. Ho sakta hai hamara apna object `data` ho jo apna `data.values()` method implement karta ho. Tab bhi hum us par `Object.values(data)` call kar sakte hain.

Doosra farak: `Object.*` methods "asli" array return karte hain, sirf iterable nahi. Ye mainly historical reasons se hai.

javascript

```
let user = {  
  name: "John",  
  age: 30  
};
```

- `Object.keys(user) = ["name", "age"]`
- `Object.values(user) = ["John", 30]`
- `Object.entries(user) = [ ["name","John"], ["age",30] ]`

`Object.values` se property values par loop:

javascript

```
let user = {  
  name: "John",  
  age: 30  
};  
  
// values par loop  
for (let value of Object.values(user)) {  
  alert(value); // John, phir 30  
}
```

`Object.keys/values/entries` symbolic properties ko ignore karte hain. `for..in` loop ki tarah ye `Symbol(...)` ko key banane wali properties ko ignore karte hain. Aam taur par ye convenient hai. Lekin symbolic keys bhi chahiye to:

- `Object.getOwnPropertySymbols`: sirf symbolic keys ka array return karta hai
- `Reflect.ownKeys(obj)`: saari keys return karta hai

2. Objects ko transform karna

Objects me arrays ke bahut se methods (`map`, `filter`, etc.) nahi hote.

Unhe lagana ho to `Object.entries` aur phir `Object.fromEntries` use kar sakte hain:

1. `Object.entries(obj)` se `obj` ke key/value pairs ka array lo.
2. Us array par array methods (jaise `map`) lagakar key/value pairs ko transform karo.
3. Result array par `Object.fromEntries(array)` lagakar wapas object banao.

Example: prices ka object hai aur hum unhe double karna chahte hain:

javascript

```

let prices = {
  banana: 1,
  orange: 2,
  meat: 4,
};

let doublePrices = Object.fromEntries(
  // prices ko array me badlo, har key/value pair ko dusre pair me map karo
  // aur fromEntries wapas object de deta hai
  Object.entries(prices).map(entry => [entry[0], entry[1] * 2])
);

alert(doublePrices.meat); // 8

```

Pehli nazar me mushkil lag sakta hai, lekin ek-do baar use karne par samajh aa jata hai. Is tarah transforms ki **powerful chains** bana sakte hain.

Practice Tasks (Answers)

1. Properties ka sum (sumSalaries): `Object.values` aur `for..of` loop se `salaries` ke saare salaries ka sum return karo. Object khali ho to `0`.

```

javascript

function sumSalaries(salaries) {

  let sum = 0;
  for (let salary of Object.values(salaries)) {
    sum += salary;
  }

  return sum;
}

let salaries = {
  "John": 100,
  "Pete": 300,
  "Mary": 250
};

alert( sumSalaries(salaries) ); // 650

```

`reduce` se bhi ho sakta hai:

javascript

```
function sumSalaries(salaries) {  
  return Object.values(salaries).reduce((a, b) => a + b, 0) // 650  
}
```

2. Properties gino (`count`): Object me properties ki ginti return karne wala function, jitna chhota ho sake:

javascript

```
function count(obj) {  
  return Object.keys(obj).length;  
}  
  
let user = {  
  name: 'John',  
  age: 30  
};  
  
alert( count(user) ); // 2
```

(Symbolic properties ignore hoti hain, sirf "regular" properties gini jati hain.)

Quick Summary

Kaam	Kaise
Keys ka array	<code>Object.keys(obj)</code>
Values ka array	<code>Object.values(obj)</code>
<code>[key, value]</code> pairs ka array	<code>Object.entries(obj)</code>
Pairs se object	<code>Object.fromEntries(pairs)</code>
Properties gino	<code>Object.keys(obj).length</code>
Object ki values par loop	<code>for (let v of Object.values(obj))</code>
Object par <code>map</code> / <code>filter</code>	<code>Object.fromEntries(Object.entries(obj).map(...))</code>
Symbolic keys bhi chahiye	<code>Reflect.ownKeys(obj)</code>
Sirf symbolic keys	<code>Object.getOwnPropertySymbols(obj)</code>

	Map	Object
Syntax	<code>map.keys()</code>	<code>Object.keys(obj)</code>
Return	iterable	Asli array

Yaad rakho:

- `Object.keys/values/entries` **symbols ko ignore** karte hain (`for...in` jaise).
- Ye **asli arrays** return karte hain, isliye seedha `map`, `filter`, `reduce` lag sakte hain.
- Object transform karne ka pattern: `entries` → **array method** → `fromEntries`.